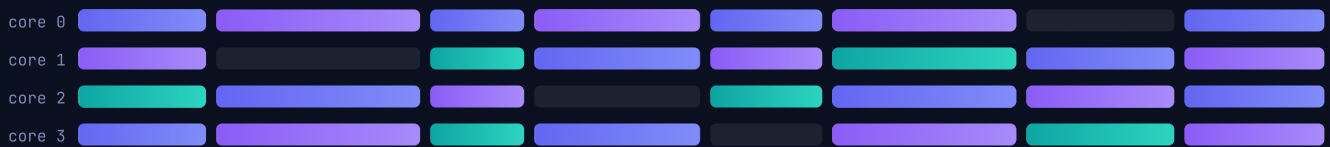


# Concurrency & Multithreading

From first principles to staff-level mastery — threads, locks, lock-free atomics, deadlocks and the mental models that win interviews at Google, Meta, Amazon, Apple & beyond.



Mutex &amp; Semaphore

Deadlock · MHNC

Race Conditions

CAS &amp; Lock-Free

Producer-Consumer

Thread Pools

Memory Model

Python GIL

# Contents

|    |                         |                                                            |
|----|-------------------------|------------------------------------------------------------|
| 01 | The Big Picture         | Why concurrency exists — the power wall & the latency wall |
| 02 | Foundations             | Concurrency vs parallelism · threads vs processes          |
| 03 | Synchronization & Locks | Race conditions, mutex vs semaphore, when to spin          |
| 04 | Deadlock                | The four Coffman conditions & how to break them            |
| 05 | Classic Problems        | Producer–consumer & reader–writer, solved                  |
| 06 | Going Lock-Free & Fast  | CAS, ABA, memory visibility, false sharing, the GIL        |
| 07 | Analogies & Mnemonics   | Memory hooks that survive interview pressure               |
| 08 | The Interview Room      | Top questions, follow-ups & good-vs-great answers          |
| 09 | Cheat Sheet             | Ten-minute revision & the ranked must-knows                |

## ◆ EDITOR'S NOTE — HOW TO READ THIS

Read cover-to-cover for genuine understanding, or jump straight to **The Interview Room** and the **Cheat Sheet** for last-minute revision. Every concept is paired with a **real-world system**, an **analogy**, and the **exact follow-up** an interviewer will ask next.



## The One Idea That Ties It All Together

Concurrency is the art of making progress on many things at once. Parallelism is the luxury of **actually** doing them at the same instant. The first is a property of your **design**; the second is a property of your **hardware**. Almost every bug, primitive and interview question in this issue flows from one uncomfortable truth — **shared mutable state accessed without coordination is undefined behavior waiting to happen.**

### 2004

THE POWER WALL

clock speeds stalled → cores multiplied

### $10^4$ – $10^6\times$

I/O VS CPU LATENCY

overlap waiting → throughput soars

### 20×

AMDAHL CEILING @ 5% SERIAL

$\text{speedup} = 1 / (S + (1-S)/N)$

02

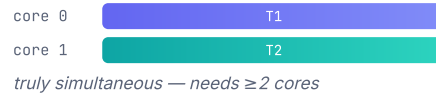
# Concurrency vs Parallelism

The single most-confused distinction in systems interviews — get it crisp in one sentence.

## 🔄 Concurrency • 1 core



## ⇒ Parallelism • 2 cores



| Dimension | Concurrency                     | Parallelism                    |
|-----------|---------------------------------|--------------------------------|
| Goal      | Structure & responsiveness      | Speed & throughput             |
| Requires  | A scheduler & context switching | Multiple physical CPU cores    |
| Example   | Single-threaded event loop      | Matrix multiply across 8 cores |
| Risk      | Races, deadlocks                | False sharing, sync overhead   |

### ♦ MENTAL MODEL

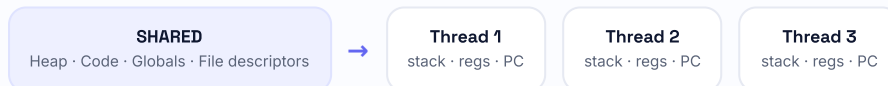
Concurrency is about **design**; parallelism is about **execution**. A single-core machine can be concurrent but **never** parallel. Say exactly this in the interview.



## Threads vs Processes

A **thread** is the smallest unit of CPU scheduling — it shares the process's heap, code and file descriptors but owns its **stack, registers and program counter**. A **process** is an isolated address space.

### ■ PROCESS MEMORY — WHAT'S SHARED VS PRIVATE



### ⚡ Use threads when...

Shared state & speed matter, or you're overlapping I/O within one task. Context switch  $\approx 100$  ns. Communication via shared memory — fast but dangerous.

### 🛡️ Use processes when...

You need isolation (a crash mustn't kill the parent), security boundaries, or different runtimes. Switch costs more; comms need IPC. `nginx` worker processes.

### ♦ ANALOGY

**Thread** = a line cook at a station: shares the kitchen, keeps their own tickets and prep. **Process** = a separate restaurant: its own kitchen and pantry, reachable only by phone order (IPC).

# The Cost of Being Correct

Coordination isn't free — pick the cheapest primitive that still guarantees safety.

A **race condition** exists whenever correctness depends on thread scheduling. It needs exactly three ingredients: **shared mutable state**, **at least one writer**, and **no synchronization**. Remove any one and the race disappears.

RACE

```
// Lost update — a textbook race condition
T1: READ counter (=0)
T2: READ counter (=0) // both read 0
T1: counter = 0+1 → WRITE 1
T2: counter = 0+1 → WRITE 1 // T1's update vanishes
// result = 1, expected = 2
```

SYNCHRONIZATION COST LADDER — CHEAP → EXPENSIVE



Rule of thumb: spin only when the critical section is shorter than a context switch (~1 μs).



## Mutex vs Semaphore vs Monitor

### Mutex

**Binary lock with ownership.** Only the acquiring thread may release. Waiters sleep. The default tool for mutual exclusion. Re-locking yourself → deadlock.

### Semaphore

**A counter (0...N), no ownership.** Any thread can signal. Perfect for resource pools and producer→consumer signalling.

### Monitor

**Mutex + condition variables** bundled with data. Java's `synchronized` is a monitor. One thread inside; others `wait()` / `notify()`.

JAVA

```
Semaphore pool = new Semaphore(10); // 10 DB connections

void queryDB() {
    pool.acquire(); // blocks if 10 already in use
    try { /* use connection */ }
    finally { pool.release(); } // ANY thread may release
}
```

### ▲ INTERVIEW TRAP

Forget to release a mutex (an exception mid-critical-section) and every other thread blocks forever. **Always** use `RAII` / `try-finally` / `with lock:`.

### ★ INTERVIEW TAKEAWAY

"Mutex has **ownership**, a semaphore is just a **counter**." You can build a mutex from a binary semaphore — but you lose priority inheritance & ownership checks.

04

# The Four Conditions — MHNC

All four must hold simultaneously. Break **any single one** and deadlock becomes impossible.

M

## Mutual Exclusion

A resource can't be shared — one holder at a time.

Break it → Prefer shareable / read-only data

H

## Hold & Wait

Hold one resource while blocking on another.

Break it → Acquire all locks atomically, up front

N

## No Preemption

A held resource can't be forcibly reclaimed.

Break it → Allow rollback & retry (DBs do this)

C

## Circular Wait

A cycle of threads each waiting on the next.

Break it → **Impose a global lock order** — the practical fix

### CIRCULAR WAIT — THE CYCLE TO HUNT FOR

Thread 1  
holds A · wants B



Thread 2  
holds B · wants A

↻ both block forever



## Breaking it in practice

### 1 • Lock ordering

Acquire locks in one global order everywhere. Kills **circular wait**. The most common real-world fix.

### 2 • Lock timeouts

`tryLock(timeout)` — give up, back off, retry. Breaks **no-preemption**.

### 3 • Detection

Build a **wait-for graph**; find cycles. Java thread dumps & DB lock monitors do this; DBs kill a victim.

### ◆ LIVELOCK

Threads stay **active** but make no progress — two cooks in the doorway side-stepping the same way forever. Fix with **randomized backoff** (exactly what Ethernet CSMA/CD does).



JAVA

```
// Lock order fixes everything: always A before B
lockA.lock();
lockB.lock();
// ... use both resources ...
lockB.unlock();
lockA.unlock();
// No circular wait can ever form → no deadlock
```

### ✦ ANALOGY

Cook A grips the only pan and waits for the oven; Cook B owns the oven and waits for the pan. Both freeze — until one kitchen rule (always grab the pan *before* the oven = **lock ordering**) makes the standoff impossible.

### ◆ STARVATION

One thread is perpetually skipped while others progress. Not deadlock. Fix with **fair FIFO** queuing — `new ReentrantLock(true)`.

05

# Producer–Consumer & Reader–Writer

The two synchronization set-pieces every interviewer keeps in their back pocket.

■ BOUNDED BUFFER — TWO SEMAPHORES GUARD THE COUNTS, ONE MUTEX GUARDS THE DATA



```
PYTHON

empty = Semaphore(CAPACITY) # free slots
full  = Semaphore(0)        # items ready
mutex = Semaphore(1)        # buffer guard

def producer(item):
    empty.acquire()          # 1) wait for a slot
    mutex.acquire()          # 2) THEN enter critical section
    buffer.append(item)
    mutex.release()
    full.release()           # signal: one more item

def consumer():
    full.acquire()
    mutex.acquire()
    item = buffer.pop(0)
    mutex.release()
    empty.release()          # signal: one more slot
```

## ▲ THE #1 MISTAKE

Order matters: acquire the **resource** semaphore ( `empty` / `full` ) **before** the `mutex` — never after. Reverse it and a full buffer deadlocks instantly.



## Reader–Writer — many readers *or* one writer



```
JAVA

ReadWriteLock rw = new ReentrantReadWriteLock();
Lock r = rw.readLock(), w = rw.writeLock();

void read() { r.lock(); try { /* many at once */ } finally { r.unlock(); } }
void write() { w.lock(); try { /* exclusive */ } finally { w.unlock(); } }
```

## ★ SENIOR INSIGHT

A `ReadWriteLock` only wins when reads dominate (>90%). Under heavy writes, the bookkeeping makes it **slower** than a plain mutex. `StampedLock` adds optimistic reads for even better throughput when conflicts are rare.

06

# CAS, ABA & the Memory Model

Where senior and staff interviews actually live — no locks, just hardware guarantees.

**Compare-And-Swap** is the atom of lock-free code: read a value, compare to expected, write the new value **only** if it still matches — all in one uninterruptible instruction ( `CMPXCHG` ).

```

JAVA
AtomicInteger counter = new AtomicInteger(0);
int cur, next;
do {
    cur = counter.get();
    next = cur + 1;
} while (!counter.compareAndSet(cur, next)); // retry if it changed
  
```

## ▲ THE ABA PROBLEM

CAS sees **A**, succeeds — but in between another thread did  $A \rightarrow B \rightarrow A$ . Nothing "changed" by value, yet the world moved. Classic in lock-free lists (node freed & reallocated at the same address).

Fix: attach a version stamp — `AtomicStampedReference`.

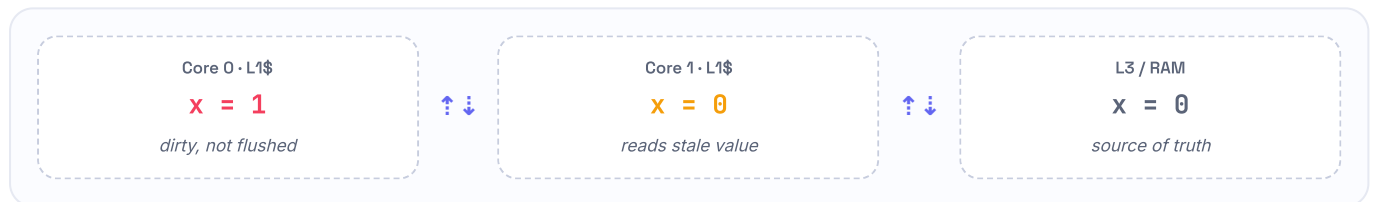
## ◆ LOCK-FREE VS WAIT-FREE

**Lock-free:** *some* thread always progresses. **Wait-free:** *every* thread progresses in bounded steps — a stronger, harder guarantee.

## 🧠 Memory Visibility — the bug locks don't obviously fix

Each core caches independently. A write on Core 0 may be invisible to Core 1 for a while — a **visibility** problem, distinct from a race.

`volatile` (Java) forces a flush on write and a fresh read, plus ordering (happens-before). But it is **not** atomic — `count++` still needs an atomic or a lock.



### False Sharing

Two threads write different variables in the **same 64-byte cache line** → constant invalidation. Fix: **pad to 64 bytes** (LMAX Disruptor's whole design).

### 🐍 Python GIL

One thread runs Python bytecode at a time. Threads help **I/O**, not CPU. For CPU parallelism use `multiprocessing` or C extensions (NumPy).

### ⚙️ Thread Pools

Reuse N workers off a queue. Size: **CPU-bound** → **cores+1**; **I/O-bound** → **cores × (1+wait/compute)**. Bound the queue or risk a memory leak.

## cores + 1

CPU-BOUND POOL SIZE

# threads ≈ cores + 1

## cores × (1+W/C)

I/O-BOUND POOL SIZE

8 cores, 90% wait →  $8 \times (1+9) = 80$

07

# Analogies & Mnemonics

Under pressure you won't recall definitions — you'll recall pictures. Here are the ones that work.



## Mutex

The single razor-sharp chef's knife — only one cook holds it; the rest wait their turn to chop.



## Semaphore

The stove's 4 burners — at most 4 dishes cook at once; free a burner and the next cook claims it.



## Deadlock

Cook A holds the only pan and wants the oven; Cook B holds the oven and wants the pan — both freeze.



## Livelock

Two cooks in the narrow doorway keep stepping the same way to yield — polite, but nobody passes.



## Race Condition

Two cooks grab the last egg for different orders — one ticket goes out wrong.



## Condition Var

The expo bell — cooks rest until 'order up!' rings, then wake and plate.



## CAS

'Is table 5's ticket still unclaimed?' — check the rail before grabbing; if taken, try the next.



## Thread Pool

A fixed brigade pulling tickets off the rail — idle cooks wait, finished cooks grab the next.



## False Sharing

Two cooks elbow over one cutting board for different veg — both slow, without truly sharing.



## Python GIL

A tiny kitchen with one stove — hire ten cooks, but only one can actually cook at a time.

## The Mnemonics Worth Memorizing

### MHNC

Deadlock's four conditions · "Make Him Never Circulate"

- M · Mutual exclusion
- H · Hold & wait
- N · No preemption
- C · Circular wait

### SLVJ

Java happens-before edges

- S · thread Start
- L · Lock release→acquire
- V · Volatile write→read
- J · thread Join

### P / V

Semaphore ops

- P · acquire — wait in the *Post* queue
- V · release — *Vacate*, next!
- CAS for counters · Lock for compounds

### ★ ONE-LINERS TO DEPLOY VERBATIM

- "volatile = visibility + ordering, not atomicity." • "Acquire the resource semaphore **before** the mutex." • "Always `while`, never `if`, around a wait." • "Pad to 64 to kill false sharing."

08

# Questions, Follow-ups & What Separates Great

The exact prompts FAANG panels reuse — with the next question they'll fire.

## Q What is a race condition — and how do you prevent it?

Output depends on thread scheduling. Needs shared mutable state,  $\geq 1$  writer, no sync. Prevent by protecting the critical section (mutex), using lock-free atomics, or eliminating sharing (immutability / thread-local).

**Follow-up** → Can you race on a single CPU? → **Yes** — a context switch can land mid read-modify-write.

## Q Explain the four deadlock conditions and how to break each.

MHNC. Break circular wait via global lock ordering (the practical one), hold-and-wait by acquiring all locks at once, no-preemption via tryLock+timeout.

**Follow-up** → How does a database detect deadlock? → Wait-for graph cycle detection, then kills the youngest/cheapest victim.

## Q Mutex vs semaphore?

Mutex: binary, has ownership (acquirer must release), for mutual exclusion. Semaphore: counter 0–N, no ownership (any thread signals), for resource counting & signalling.

**Follow-up** → Build a mutex from a semaphore? → Yes, a binary semaphore by convention — but you lose ownership checks & priority inheritance.

## Q Why volatile in a double-checked-locking singleton?

Without it, a thread can observe a *partially constructed* object — the constructor's writes can be reordered after the reference write. Volatile's happens-before forbids that.

**Follow-up** → When is volatile enough? → Single-variable publish/flag. Never for `count++` — use AtomicInteger.

## ↑ Good vs Great

| A good answer says...              | A great answer says...                                                                                  |
|------------------------------------|---------------------------------------------------------------------------------------------------------|
| "Use a mutex."                     | "Mutex here — but if reads dominate, a ReadWriteLock lifts throughput."                                 |
| "volatile makes it thread-safe."   | "volatile gives visibility & ordering, not compound atomicity."                                         |
| "Threads run concurrently."        | "On N cores, up to N run in parallel; the rest context-switch."                                         |
| "Use a thread pool."               | "I/O-bound → $\text{cores} \times (1 + \text{wait}/\text{compute})$ ; CPU-bound → $\text{cores} + 1$ ." |
| "Deadlock is two threads waiting." | "Four Coffman conditions; break circular wait with lock ordering."                                      |

### ◆ HOW CONCURRENCY CONNECTS TO THE REST OF YOUR LOOP

**OS:** futures, scheduling, copy-on-write fork. **System Design:** connection pools = semaphores, Kafka = distributed producer-consumer, Redis `SET NX` = distributed mutex. **Databases:** `SELECT FOR UPDATE` = pessimistic lock, version columns = CAS, MVCC = the reader-writer solution. **Distributed:** Raft/Paxos = a distributed condition variable.

09

# The Cheat Sheet

Everything in this issue, compressed for the morning of the interview.

## ① RACE CONDITIONS

Shared mutable state + no sync → lock or atomics, or remove sharing.

## ② DEADLOCKS

MHNC conditions → break circular wait with **lock ordering**.

## ③ VISIBILITY

CPU caches lie → volatile for flags, atomic/lock for compounds.

| Concept                  | One-liner                           | Key code / formula                   | Gotcha                            |
|--------------------------|-------------------------------------|--------------------------------------|-----------------------------------|
| <b>Mutex</b>             | Binary lock with ownership          | <code>lock()...unlock()</code>       | Forgetting release = deadlock     |
| <b>Semaphore</b>         | Counter lock, any thread signals    | <code>wait()...signal()</code>       | Acquire resource sem before mutex |
| <b>Condition Var</b>     | Sleep & release lock atomically     | <code>while(!cond) wait()</code>     | Use while, not if                 |
| <b>volatile</b>          | Visibility + ordering, no atomicity | <code>volatile boolean ready</code>  | Not enough for count++            |
| <b>CAS / Atomic</b>      | HW read-modify-write                | <code>compareAndSet(old,new)</code>  | The ABA problem                   |
| <b>Spinlock</b>          | Busy-wait loop                      | <code>while(test_and_set()){}</code> | Burns CPU; <1 μs only             |
| <b>Deadlock</b>          | Circular wait for resources         | conditions: MHNC                     | Prevent via lock ordering         |
| <b>Thread Pool</b>       | Pre-made workers + queue            | <code>cores+1 / cores×10</code>      | Unbounded queue = leak            |
| <b>False Sharing</b>     | Diff vars, same cache line          | pad to 64 bytes                      | Needs perf tools to spot          |
| <b>Python GIL</b>        | One bytecode thread at a time       | <code>multiprocessing</code> for CPU | I/O threads still help            |
| <b>Producer-Consumer</b> | Bounded-buffer sync                 | <code>empty+full</code> sems + mutex | Order: sem → mutex                |
| <b>async/await</b>       | Cooperative coroutines              | <code>await gather(*tasks)</code>    | Sync code blocks the loop         |



## Most-asked, ranked

|    |                                        |                   |
|----|----------------------------------------|-------------------|
| 01 | Race condition: define + example + fix | EVERYWHERE        |
| 02 | Deadlock — MHNC + prevention           | SENIOR+           |
| 03 | Producer-consumer implementation       | AMAZON · GOOGLE   |
| 04 | volatile vs synchronized vs atomic     | JAVA SHOPS        |
| 05 | Thread-pool sizing                     | SYS-DESIGN        |
| 06 | Mutex vs semaphore                     | UNIVERSAL         |
| 07 | False sharing                          | PERF-HEAVY        |
| 08 | Condition var + while-loop             | DEPTH SIGNAL      |
| 09 | Python GIL                             | PYTHON SHOPS      |
| 10 | CAS & the ABA problem                  | STAFF / LOCK-FREE |



## If you have 60 seconds

### ★ SAY THESE FIVE THINGS

- 1 · Concurrency = design, parallelism = execution.
- 2 · A race needs shared state + a writer + no sync.
- 3 · Deadlock = MHNC; fix with lock ordering.
- 4 · Mutex owns; semaphore counts.
- 5 · volatile ≠ atomic; CAS for counters, locks for compounds.

### ✦ LANGUAGE QUICK-FACTS

**Java:** synchronized = monitor; Atomic\* = CAS; virtual threads (21).

**Python:** GIL → multiprocessing for CPU.

**Go:** prefer channels; goroutines ≈ 2 KB stacks; `go test -race`.

ENGINEERING ESSENTIALS

# Now go break the circular wait.

Master one primitive at a time. Understand the hardware beneath it. Always reach for the **cheapest** tool that's still correct.

Vol. 01 — Concurrency & Multithreading

FAANG Interview Master Series

*Designed for deep study & 10-minute revision · A4 · Self-contained*